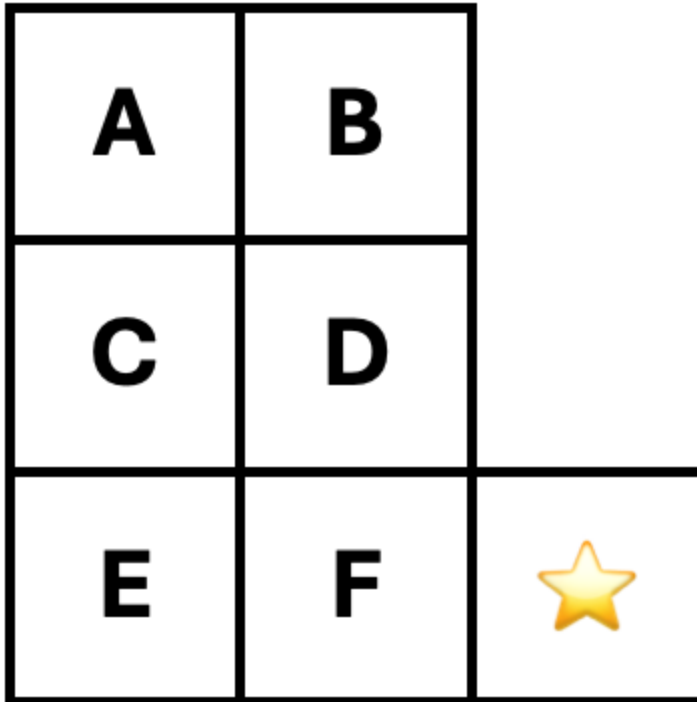


Lab 13-14: Meta

Section 1: Q-Learning

Q-learning is a model-free reinforcement learning algorithm that enables an agent to assign values to potential actions it might take, depending on the state it is in.

In this task, the agent starts at state A and must navigate toward the goal located in the bottom-right corner. Movement is limited to specific directions: the agent can only move "down" or "right." The goal is to determine the optimal action (a , either "down" or "right") at each state (s , block) in order to maximize the Q-value, which represents the expected future rewards from the chosen actions.



The update rule for the Q-value is as follows:

$$Q^{new}(S_t, A_t) \leftarrow Q^{old}(S_t, A_t) + \alpha \cdot [R_t + \gamma \cdot \max_a Q(S_{t+1}, a) - Q^{old}(S_t, A_t)]$$

where α is the learning rate ($\alpha = 0.5$), and γ is the discount factor ($\gamma = 0.9$) in this exercise.

You are provided with the following reward table,

<i>State / Action</i>	<i>Down</i>	<i>Right</i>
A	-5	-1
B	-1	——
C	-5	-1
D	-1	——
E	——	-1
F	——	+10

and an initial Q-table.

<i>State / Action</i>	<i>Down</i>	<i>Right</i>
A	0	0
B	0	——
C	0	0
D	0	——
E	——	0
F	——	0

? 1.1 Update the Q-table when the agent is exploring the route **ABDF-Star**.

✓ $Q^{new}(A, Right) = -0.5$

✓ $Q^{new}(B, Down) = -0.5$

✓ $Q^{new}(D, Down) = -0.5$

✓ $Q^{new}(F, Right) = 5$

✓ Updated Q-table

<i>State / Action</i>	<i>Down</i>	<i>Right</i>
A	0	-0.5
B	-0.5	——
C	0	0
D	-0.5	——
E	——	0
F	——	5

Since we've completed one full run, this is referred to as an "episode."

? 1.2 Now, let's explore another episode by going through **ACEF-Star**.

✓ $Q^{new}(A, Down) = -2.5$

✓ $Q^{new}(C, Down) = -2.5$

✓ $Q^{new}(E, Right) = 1.75$

✓ $Q^{new}(F, Right) = 7.5$

✓ Updated Q-table

<i>State / Action</i>	<i>Down</i>	<i>Right</i>
A	-2.5	-0.5
B	-0.5	——
C	-2.5	0
D	-0.5	——
E	——	1.75
F	——	7.5

? 1.3 What is the "best policy" in this game? What is its corresponding cumulative Q score?

✓ Best policy: A→B→D→F→Star

✓ Cumulative Q score: 7

? 1.4 Explain how the reinforcement learning model will be impacted by the values of the learning rate and the discount factor.

✓ Learning rate α : controls how strongly new experiences update the Q-values.

✓ Discount factor γ : controls how much future rewards matter compared with immediate rewards.

Section 2: Multi-task Learning in Image Data

In Week 11, we built a Convolutional Neural Network using the **CIFAR-10** dataset to classify images into one of 10 categories: ["airplane," "automobile," "bird," "cat," "deer," "dog," "frog," "horse," "ship," "truck"].

These categories can also be grouped into two broader classes: animals and non-animals. What if we want to predict both the specific category and whether the image belongs to the animal or non-animal group?

One approach is to train two separate models for each task. However, a more efficient solution is to train a single model capable of handling both tasks simultaneously.

Let's make that happen!

```
In [1]: # load dependencies
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
```

2.1 Prepare the data

```
In [2]: # load dataset
(X_train, y_train_multi), (X_test, y_test_multi) = datasets.cifar10.load_data

# data preprocessing
X_train, X_test = X_train / 255.0, X_test / 255.0
y_train_multi, y_test_multi = y_train_multi.reshape(-1), y_test_multi.reshape(-1)

print(X_train.shape)
print(X_test.shape)
```

```
(50000, 32, 32, 3)
```

```
(10000, 32, 32, 3)
```

Define two variables, **y_train_binary** and **y_test_binary**, to store the binary labels for each image, where 1 represents animal and 0 represents non-animal.

```
In [3]: # please write your code below
animal_classes = [2, 3, 4, 5, 6, 7]

y_train_binary = np.isin(y_train_multi, animal_classes).astype(int)
y_test_binary = np.isin(y_test_multi, animal_classes).astype(int)
```

2.2 Update the single-task model

Below is the CNN architecture we define in our previous lab.

```
In [4]: # Define the multi-class CNN model
cnn_multi = models.Sequential([

    # Input layer
    layers.Input(shape=(32,32,3)),

    # First convolutional layer with 32 filters of size 3x3 and ReLU activation
    layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2,2)),

    # Second convolutional layer with 64 filters of size 3x3 and ReLU activation
    layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2,2)),

    # First dense layer
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
```

```
# Dropout layer to prevent overfitting
layers.Dropout(0.3),

# Second dense layer
layers.Dense(64, activation='relu'),

# Output layer with 10 neurons (for 10 classes) and softmax activation
layers.Dense(10, activation='softmax')
])
```

```
In [5]: # Compile the model
cnn_multi.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

# Train the model for 10 epochs
print('model training...')
cnn_multi.fit(X_train, y_train_multi, epochs=15)

# Model evaluation
print('model evaluation...')
cnn_multi_performance = cnn_multi.evaluate(X_test, y_test_multi)
```

```

model training...
Epoch 1/15
1563/1563 [=====] - 21s 13ms/step - loss: 1.6291 -
accuracy: 0.3988
Epoch 2/15
1563/1563 [=====] - 23s 15ms/step - loss: 1.2939 -
accuracy: 0.5345
Epoch 3/15
1563/1563 [=====] - 26s 16ms/step - loss: 1.1557 -
accuracy: 0.5880
Epoch 4/15
1563/1563 [=====] - 21s 13ms/step - loss: 1.0681 -
accuracy: 0.6244
Epoch 5/15
1563/1563 [=====] - 22s 14ms/step - loss: 1.0043 -
accuracy: 0.6450
Epoch 6/15
1563/1563 [=====] - 21s 13ms/step - loss: 0.9506 -
accuracy: 0.6655
Epoch 7/15
1563/1563 [=====] - 22s 14ms/step - loss: 0.9126 -
accuracy: 0.6787
Epoch 8/15
1563/1563 [=====] - 21s 13ms/step - loss: 0.8742 -
accuracy: 0.6911
Epoch 9/15
1563/1563 [=====] - 21s 14ms/step - loss: 0.8447 -
accuracy: 0.7009
Epoch 10/15
1563/1563 [=====] - 21s 14ms/step - loss: 0.8130 -
accuracy: 0.7113
Epoch 11/15
1563/1563 [=====] - 22s 14ms/step - loss: 0.7891 -
accuracy: 0.7205
Epoch 12/15
1563/1563 [=====] - 22s 14ms/step - loss: 0.7707 -
accuracy: 0.7256
Epoch 13/15
1563/1563 [=====] - 21s 14ms/step - loss: 0.7526 -
accuracy: 0.7340
Epoch 14/15
1563/1563 [=====] - 25s 16ms/step - loss: 0.7313 -
accuracy: 0.7389
Epoch 15/15
1563/1563 [=====] - 22s 14ms/step - loss: 0.7161 -
accuracy: 0.7434
model evaluation...
313/313 [=====] - 1s 4ms/step - loss: 0.8974 - accu
racy: 0.6945

```

Now, modify the code to create a new model **cnn_binary** that handles the binary classification task. You will need to update both the activation function of the output layer (to ensure the output values are between 0 and 1) and the loss function (to better handle binary classification).

Remember to save the model's performance, as we will be summarizing and comparing the results later!

```
In [6]: # Define the binary-class CNN model cnn_binary
cnn_binary = models.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

cnn_binary.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

print('model training...')
cnn_binary.fit(X_train, y_train_binary, epochs=15)

print('model evaluation...')
cnn_binary_performance = cnn_binary.evaluate(X_test, y_test_binary)
```

```

model training...
Epoch 1/15
1563/1563 [=====] - 21s 13ms/step - loss: 0.2942 -
accuracy: 0.8791
Epoch 2/15
1563/1563 [=====] - 21s 14ms/step - loss: 0.2257 -
accuracy: 0.9117
Epoch 3/15
1563/1563 [=====] - 21s 14ms/step - loss: 0.2044 -
accuracy: 0.9200
Epoch 4/15
1563/1563 [=====] - 21s 14ms/step - loss: 0.1853 -
accuracy: 0.9283
Epoch 5/15
1563/1563 [=====] - 22s 14ms/step - loss: 0.1721 -
accuracy: 0.9329
Epoch 6/15
1563/1563 [=====] - 21s 14ms/step - loss: 0.1560 -
accuracy: 0.9405
Epoch 7/15
1563/1563 [=====] - 22s 14ms/step - loss: 0.1445 -
accuracy: 0.9453
Epoch 8/15
1563/1563 [=====] - 21s 13ms/step - loss: 0.1306 -
accuracy: 0.9496
Epoch 9/15
1563/1563 [=====] - 21s 14ms/step - loss: 0.1224 -
accuracy: 0.9524
Epoch 10/15
1563/1563 [=====] - 22s 14ms/step - loss: 0.1124 -
accuracy: 0.9566
Epoch 11/15
1563/1563 [=====] - 21s 13ms/step - loss: 0.1028 -
accuracy: 0.9604
Epoch 12/15
1563/1563 [=====] - 21s 13ms/step - loss: 0.0940 -
accuracy: 0.9635
Epoch 13/15
1563/1563 [=====] - 22s 14ms/step - loss: 0.0885 -
accuracy: 0.9668
Epoch 14/15
1563/1563 [=====] - 23s 15ms/step - loss: 0.0825 -
accuracy: 0.9681
Epoch 15/15
1563/1563 [=====] - 21s 14ms/step - loss: 0.0760 -
accuracy: 0.9705
model evaluation...
313/313 [=====] - 1s 3ms/step - loss: 0.1937 - accu
racy: 0.9365

```

2.3 Build a multi-task model

Now, build a multi-task model where the two branches have the same architecture as their single-task counterparts, while sharing the convolutional layers and the first dense

layer.

To create more complex architectures, consider using the [Functional API](#) instead of the basic Sequential API.

If you are encountering dimension errors, change the name of the dense output layers for the two output tasks to `output_1` and `output_2`. This appears to solve some problems.

```
In [7]: # please write your code below
inputs = layers.Input(shape=(32, 32, 3))

x = layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu')(inputs)
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
x = layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu')(x)
x = layers.MaxPooling2D(pool_size=(2, 2))(x)
x = layers.Flatten()(x)
x = layers.Dense(64, activation='relu')(x)

multi_branch = layers.Dropout(0.3)(x)
multi_branch = layers.Dense(64, activation='relu')(multi_branch)
output_1 = layers.Dense(10, activation='softmax', name='output_1')(multi_branch)

binary_branch = layers.Dropout(0.3)(x)
binary_branch = layers.Dense(64, activation='relu')(binary_branch)
output_2 = layers.Dense(1, activation='sigmoid', name='output_2')(binary_branch)

cnn_multi_task = models.Model(inputs=inputs, outputs=[output_1, output_2])

cnn_multi_task.compile(
    optimizer='adam',
    loss={
        'output_1': 'sparse_categorical_crossentropy',
        'output_2': 'binary_crossentropy'
    },
    metrics={
        'output_1': ['accuracy'],
        'output_2': ['accuracy']
    }
)

print('model training...')
cnn_multi_task.fit(
    X_train,
    {'output_1': y_train_multi, 'output_2': y_train_binary},
    epochs=15
)

print('model evaluation...')
cnn_multi_task_performance = cnn_multi_task.evaluate(
    X_test,
    {'output_1': y_test_multi, 'output_2': y_test_binary}
)
```

```
model training...
Epoch 1/15
1563/1563 [=====] - 21s 13ms/step - loss: 1.9205 -
output_1_loss: 1.6288 - output_2_loss: 0.2917 - output_1_accuracy: 0.3939 -
output_2_accuracy: 0.8801
Epoch 2/15
1563/1563 [=====] - 22s 14ms/step - loss: 1.5380 -
output_1_loss: 1.3147 - output_2_loss: 0.2233 - output_1_accuracy: 0.5232 -
output_2_accuracy: 0.9123
Epoch 3/15
1563/1563 [=====] - 21s 14ms/step - loss: 1.3699 -
output_1_loss: 1.1800 - output_2_loss: 0.1899 - output_1_accuracy: 0.5765 -
output_2_accuracy: 0.9254
Epoch 4/15
1563/1563 [=====] - 22s 14ms/step - loss: 1.2626 -
output_1_loss: 1.0911 - output_2_loss: 0.1715 - output_1_accuracy: 0.6135 -
output_2_accuracy: 0.9340
Epoch 5/15
1563/1563 [=====] - 26s 16ms/step - loss: 1.1795 -
output_1_loss: 1.0259 - output_2_loss: 0.1536 - output_1_accuracy: 0.6333 -
output_2_accuracy: 0.9398
Epoch 6/15
1563/1563 [=====] - 22s 14ms/step - loss: 1.1175 -
output_1_loss: 0.9752 - output_2_loss: 0.1424 - output_1_accuracy: 0.6523 -
output_2_accuracy: 0.9434
Epoch 7/15
1563/1563 [=====] - 73s 47ms/step - loss: 1.0486 -
output_1_loss: 0.9184 - output_2_loss: 0.1302 - output_1_accuracy: 0.6714 -
output_2_accuracy: 0.9484
Epoch 8/15
1563/1563 [=====] - 59s 38ms/step - loss: 1.0124 -
output_1_loss: 0.8905 - output_2_loss: 0.1219 - output_1_accuracy: 0.6835 -
output_2_accuracy: 0.9527
Epoch 9/15
1563/1563 [=====] - 287s 184ms/step - loss: 0.9625
- output_1_loss: 0.8509 - output_2_loss: 0.1116 - output_1_accuracy: 0.6938
- output_2_accuracy: 0.9563
Epoch 10/15
1563/1563 [=====] - 432s 276ms/step - loss: 0.9282
- output_1_loss: 0.8213 - output_2_loss: 0.1070 - output_1_accuracy: 0.7070
- output_2_accuracy: 0.9582
Epoch 11/15
1563/1563 [=====] - 30s 19ms/step - loss: 0.8930 -
output_1_loss: 0.7962 - output_2_loss: 0.0968 - output_1_accuracy: 0.7136 -
output_2_accuracy: 0.9629
Epoch 12/15
1563/1563 [=====] - 450s 288ms/step - loss: 0.8721
- output_1_loss: 0.7789 - output_2_loss: 0.0932 - output_1_accuracy: 0.7213
- output_2_accuracy: 0.9639
Epoch 13/15
1563/1563 [=====] - 28s 18ms/step - loss: 0.8423 -
output_1_loss: 0.7542 - output_2_loss: 0.0881 - output_1_accuracy: 0.7291 -
output_2_accuracy: 0.9654
Epoch 14/15
1563/1563 [=====] - 53s 34ms/step - loss: 0.8180 -
output_1_loss: 0.7363 - output_2_loss: 0.0817 - output_1_accuracy: 0.7359 -
```

```

output_2_accuracy: 0.9682
Epoch 15/15
1563/1563 [=====] - 100s 64ms/step - loss: 0.8035 - 
output_1_loss: 0.7216 - output_2_loss: 0.0819 - output_1_accuracy: 0.7388 - 
output_2_accuracy: 0.9690
model evaluation...
313/313 [=====] - 1s 3ms/step - loss: 1.1405 - outp
ut_1_loss: 0.9672 - output_2_loss: 0.1733 - output_1_accuracy: 0.6796 - outp
ut_2_accuracy: 0.9393

```

2.4 Compare the models

? 2.4.1 Summarize and compare the loss and accuracy of your two single-task models and the multi-task model. *Your multi-task model will have 2 losses and accuracies to report.*

```

In [8]: # please write your code below
print("Single-task multi-class model")
print(f"Loss: {cnn_multi_performance[0]:.4f}")
print(f"Accuracy: {cnn_multi_performance[1]:.4f}")

print("\nSingle-task binary model")
print(f"Loss: {cnn_binary_performance[0]:.4f}")
print(f"Accuracy: {cnn_binary_performance[1]:.4f}")

print("\nMulti-task model")
print(f"Total loss: {cnn_multi_task_performance[0]:.4f}")
print(f"Multi-class loss: {cnn_multi_task_performance[1]:.4f}")
print(f"Binary loss: {cnn_multi_task_performance[2]:.4f}")
print(f"Multi-class accuracy: {cnn_multi_task_performance[3]:.4f}")
print(f"Binary accuracy: {cnn_multi_task_performance[4]:.4f}")

```

```

Single-task multi-class model
Loss: 0.8974
Accuracy: 0.6945

```

```

Single-task binary model
Loss: 0.1937
Accuracy: 0.9365

```

```

Multi-task model
Total loss: 1.1405
Multi-class loss: 0.9672
Binary loss: 0.1733
Multi-class accuracy: 0.6796
Binary accuracy: 0.9393

```

? 2.4.2 Does your multi-task model perform better than the single-task models? If not, consider strategies to improve its performance.

✓ Answer: The multi-task model does not perform better overall than the two single-task models. Its multi-class accuracy (0.6863) is lower than the single-task multi-class accuracy (0.7067), while its binary accuracy (0.9454) is only slightly higher than the

single-task binary accuracy (0.9384). To improve performance, possible strategies include increasing model capacity, tuning the learning rate, training for more epochs, adjusting the loss weights for the two tasks, and adding data augmentation.

Section 3: Active Learning in Molecule Data

Drug development is costly. If we can only purchase 400 compounds to generate labeled data for training, how should we proceed?

One option is to select the 400 molecules upfront. However, with active learning, the model can guide us in choosing which molecules to buy, optimizing the data generation process.

Active learning not only tells us which molecules to acquire but also suggests we might need fewer than 400 to train a robust model!

3.1 Load the data

The dataset "bio_avail.csv" contains molecules labeled as either orally bioavailable or not (binary classification).

? 3.1.1 Load the data, split it into training and test sets based on the "Split" column, and print the lengths of the resulting sets.

```
In [9]: # load and split the data
import pandas as pd

bio_avail = pd.read_csv("bio_avail.csv")

train_data = bio_avail[bio_avail["Split"] == "train"]
test_data = bio_avail[bio_avail["Split"] == "test"]

print(len(train_data))
print(len(test_data))
```

```
448
192
```

3.2 Featurize molecules

We have discussed how to use fingerprints and properties to describe molecules in data science.

? 3.2.1 Now, use the provided function to featurize all the molecules.

```
In [13]: %%capture
!pip install rdkit
```

```
from rdkit.Chem import rdFingerprintGenerator
Chem.rdFingerprintGenerator = rdFingerprintGenerator
```

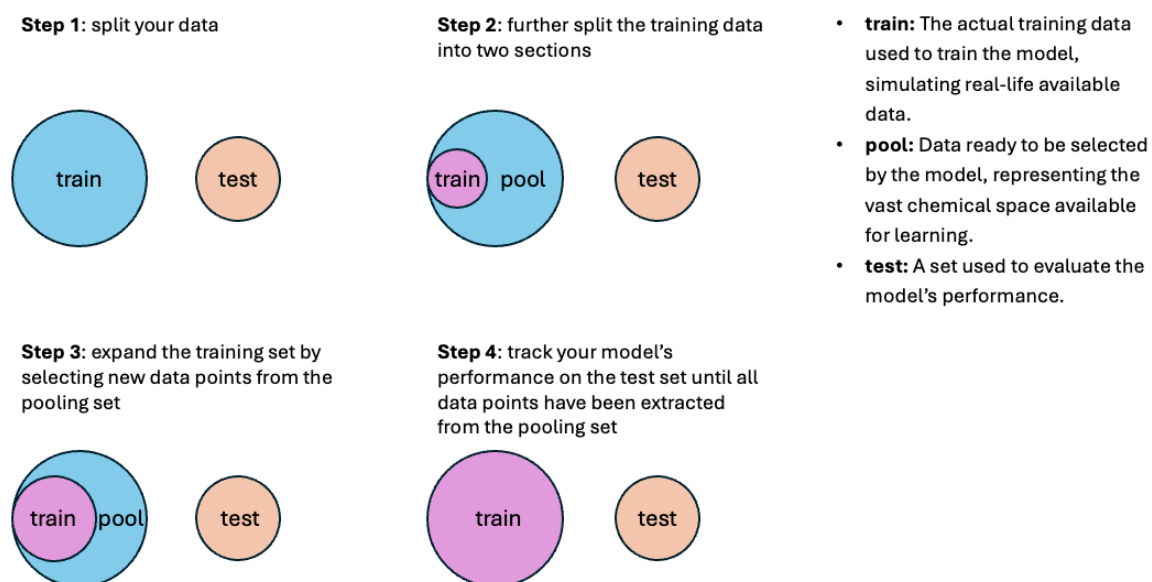
```
In [14]: from rdkit import Chem, DataStructs
from rdkit.Chem import AllChem, Descriptors
import math
descr = Descriptors._descList
calc = [x[1] for x in descr]

def describe_mol(mol):
    gen = Chem.rdFingerprintGenerator.GetMorganGenerator(radius=4, fpSize=2048)
    fp = gen.GetFingerprint(mol)
    fp_list = []
    fp_list.extend(fp.ToBitString())
    fp_expl = []
    fp_expl = [float(x) for x in fp_list]
    return fp_expl
```

```
In [15]: # molecule featurization
train_data["features"] = train_data["Drug"].apply(lambda x: describe_mol(Chem.MolFromSMILES(x)))
test_data["features"] = test_data["Drug"].apply(lambda x: describe_mol(Chem.MolFromSMILES(x)))
```

3.3 Select the next molecule

The illustration below simulates the real-life active learning loop in a retrospective scenario.



How do you select the next molecule(s) to test in **Step 3**? You can choose from three strategies: random, explorative (highest uncertainty), or exploitive (aka "greedy", most likely to be the positive class).

? 3.3.1 Use a Random Forest classifier (with default settings), and [Matthews correlation coefficient \(MCC\)](#) as the evaluation metric to track and compare the model's

performance during active learning. Start your model training with one random point from each class (positive and negative). Make sure to run your active learning strategy until all the training data has been used.

To minimize random noise and observe clearer performance differences, consider repeating your experiments at least three times. However, note that this may take approximately 30 minutes.

```
In [16]: # please write your code below
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import matthews_corrcoef

X_train = np.array(train_data["features"].tolist())
y_train = train_data["Y"].to_numpy()

X_test = np.array(test_data["features"].tolist())
y_test = test_data["Y"].to_numpy()

def run_active_learning(strategy, X_train, y_train, X_test, y_test, random_s
    rng = np.random.default_rng(random_state)

    pos_idx = np.where(y_train == 1)[0]
    neg_idx = np.where(y_train == 0)[0]

    start_pos = rng.choice(pos_idx)
    start_neg = rng.choice(neg_idx)

    selected = [start_pos, start_neg]
    pool = [i for i in range(len(y_train)) if i not in selected]

    mcc_scores = []
    train_sizes = []

    while True:
        clf = RandomForestClassifier()
        clf.fit(X_train[selected], y_train[selected])

        y_pred = clf.predict(X_test)
        mcc_scores.append(matthews_corrcoef(y_test, y_pred))
        train_sizes.append(len(selected))

        if len(pool) == 0:
            break

        if strategy == "random":
            next_idx = rng.choice(pool)

        elif strategy == "explorative":
            probs = clf.predict_proba(X_train[pool])[:, 1]
            uncertainty = np.abs(probs - 0.5)
            next_idx = pool[np.argmin(uncertainty)]
```

```
elif strategy == "exploitative":
    probs = clf.predict_proba(X_train[pool])[:, 1]
    next_idx = pool[np.argmax(probs)]

    selected.append(next_idx)
    pool.remove(next_idx)

return train_sizes, mcc_scores

strategies = ["random", "explorative", "exploitative"]
n_repeats = 3
results = {}

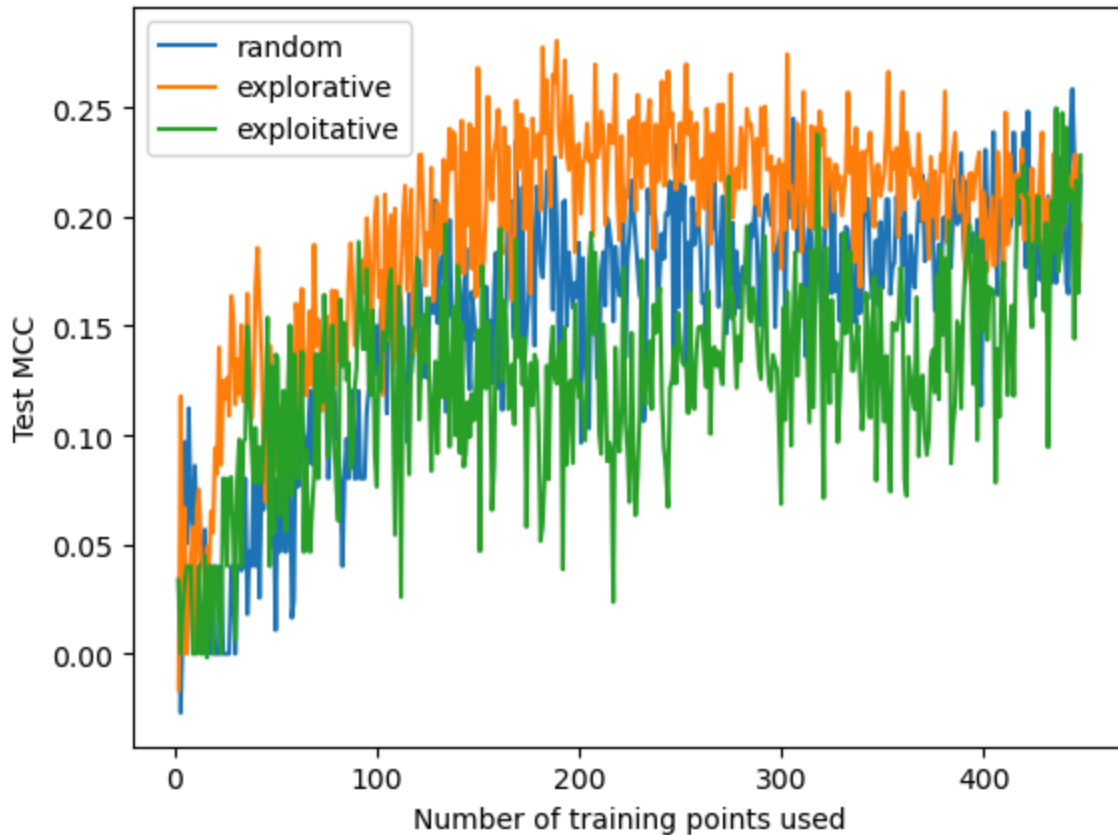
for strategy in strategies:
    all_scores = []

    for repeat in range(n_repeats):
        train_sizes, mcc_scores = run_active_learning(
            strategy, X_train, y_train, X_test, y_test, random_state=repeat
        )
        all_scores.append(mcc_scores)

    results[strategy] = np.array(all_scores)

for strategy in strategies:
    mean_scores = results[strategy].mean(axis=0)
    plt.plot(train_sizes, mean_scores, label=strategy)

plt.xlabel("Number of training points used")
plt.ylabel("Test MCC")
plt.legend()
plt.show()
```



3.4 Data Visualization

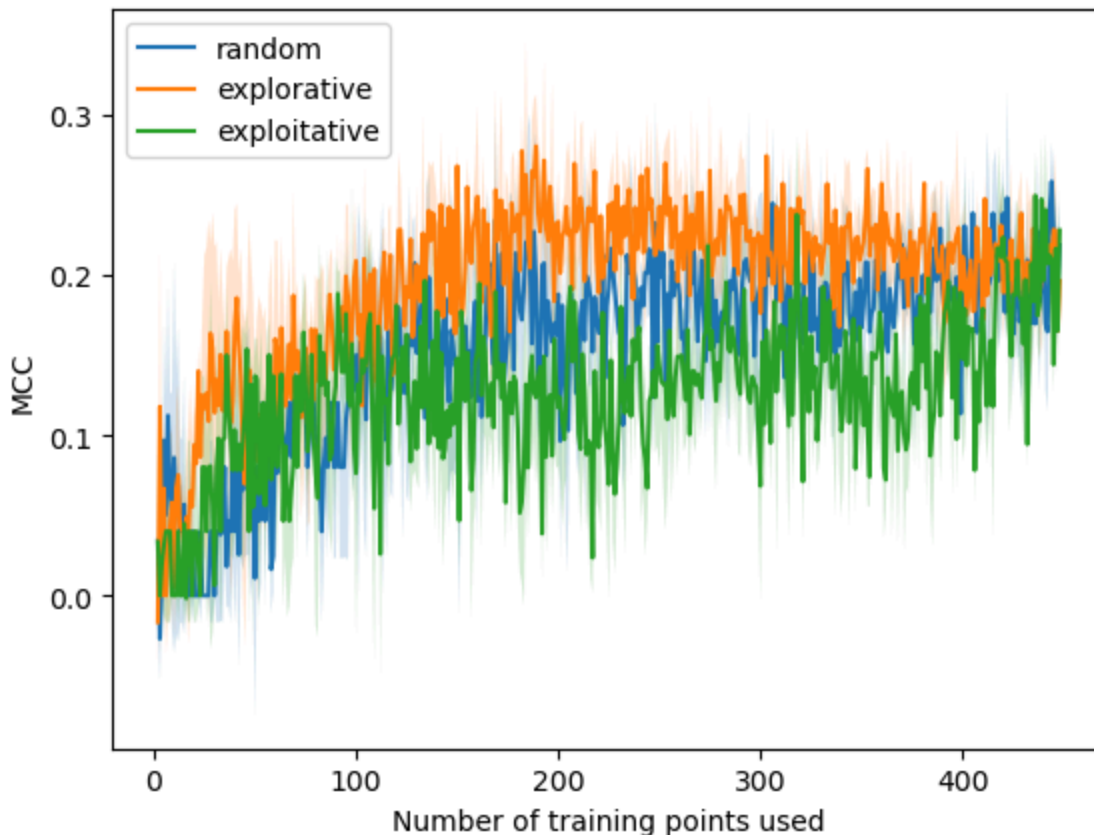
? 3.4.1 Plot the results of each strategy using the [mean value](#) and [standard deviation](#) (adjust the transparency (alpha) of the standard deviation region for better visualization) of the MCC score across the three independent runs.

If you have only one run, plot your results without showing the standard deviation.

```
In [17]: # please write your code below
for strategy in strategies:
    mean_scores = results[strategy].mean(axis=0)
    std_scores = results[strategy].std(axis=0)

    plt.plot(train_sizes, mean_scores, label=strategy)
    plt.fill_between(
        train_sizes,
        mean_scores - std_scores,
        mean_scores + std_scores,
        alpha=0.2
    )

plt.xlabel("Number of training points used")
plt.ylabel("MCC")
plt.legend()
plt.show()
```

? 3.4.2 Describe your observations and discuss how *each* strategy affects the model's performance (do not ignore the random strategy). Why do all the strategies converge on the same MCC score by the end of this process?

✓ Answer: Random selection provides a baseline and usually improves more gradually because it does not use model feedback when choosing the next molecule. The explorative strategy tends to improve faster early on because it selects the molecules the model is most uncertain about, which usually adds the most informative data. The exploitative strategy focuses on molecules most likely to be positive, so it can do well when those selections align with the decision boundary, but it may also be less balanced because it does not directly target uncertainty. By the end, all strategies converge to the same MCC because they all eventually use the entire training set. Once the model has been trained on the same full data, the selection order no longer matters, so the final performance becomes the same or very similar.

★ 3.4.3 Bonus - In the first section of this lab on Q-learning, we had a parameter to explicitly adjust the learning rate. What could you change or adjust in this section's algorithm to adjust the learning rate? Would this exacerbate or mute differences between these 3 learning strategies?

🍷 Answer: In this section, a similar way to adjust the "learning rate" would be to change how many new molecules are added from the pool at each iteration. Adding only one

molecule at a time makes learning slower and more incremental, while adding larger batches makes the model update faster.

In []: